Data Lake vs Data Warehouse vs Lakehouse

한눈에 보는 비교

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    데이터 아키텍처 진화 과정                                 │
│                                                                          │
│  1980s~2000s          2010s              2020s~                          │
│                                                                          │
│  ┌──────────────┐    ┌──────────────┐    ┌──────────────────────┐        │
│  │   Data       │    │   Data       │    │     Lakehouse        │        │
│  │  Warehouse   │    │   Lake       │    │                      │        │
│  │              │    │              │    │  ┌────────────────┐   │        │
│  │ 정형 데이터|    | 모든 데이터|    |  |   Lake 계층    |  |          |
│  │ SQL 분석  |    | 저렴한 저장|    |  |  (Raw 저장)    |  |          |
│  │ 스키마 O  |    | 스키마 X   |    |  ├────────────────┤  |          |
│  │              |    |              |    |  | Warehouse 계층|  |          |
│  │ $$$       |    | $          |    |  | (ACID, Schema)|  |          |
│  └──────────────┘    └──────────────┘    |  └────────────────┘   |        │
│                                          └──────────────────────┘        │
│       ↓                  ↓                       ↓                       │
│   BI, 리포팅          ML, 빅데이터          BI + ML 통합                 |
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

비교 표

항목	Data Warehouse	Data Lake	Lakehouse
데이터 형태	정형 데이터만	정형 + 비정형 모두	정형 + 비정형 모두
스키마	Schema-on-Write	Schema-on-Read	둘 다 지원
저장 비용	높음 ($$$)	낮음 ($)	낮음 ($)
데이터 품질	높음 (정제됨)	낮음 (Raw)	높음 (거버넌스)
ACID 트랜잭션	☑ 지원	✕ 미지원	☑ 지원
실시간 처리	제한적	☑ 가능	☑ 가능
주요 사용자	BI 분석가	데이터 과학자	모두
쿼리 성능	빠름	느림	빠름
대표 기술	Snowflake, Redshift	S3, HDFS	Delta Lake, Iceberg

Part 1: Data Warehouse (데이터 웨어하우스)

개념

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          Data Warehouse 구조                              │
│                                                                           │
│   데이터 소스들              ETL 과정              Data Warehouse          │
│                                                                           │
│   ┌─────────┐                                  ┌──────────────────────┐  │
│   │   ERP   │──┐                               │                      │  │
│   └─────────┘  │                              │  ┌────────────┐      │  │
│                │          ┌──────────────┐    │  │ 판매 테이블 │      │  │
│   ┌─────────┐  ├─────────>│              │────>│  ├────────────┤      │  │
│   │   CRM   │──┤          │   E T L      │    │  │ 고객 테이블 │      │  │
│   └─────────┘  │          │              │    │  ├────────────┤      │  │
│                │          │ Extract      │    │  │ 제품 테이블 │      │  │
│   ┌─────────┐  │          │ Transform    │    │  └────────────┘      │  │
│   │  Excel  │──┘          │ Load         │    │                      │  │
│   └─────────┘             │              │    │  정형화된 스키마      │  │
│                           └──────────────┘    │  SQL 쿼리 최적화      │  │
│   (다양한 소스)          (정제/변환)          │  빠른 분석            │  │
│                                                └──────────────────────┘  │
│                                                          │               │
│                                                          ▼               │
│                                                  ┌──────────────┐        │
│                                                  │ BI Dashboard │        │
│                                                  │ 리포트, 분석 │        │
│                                                  └──────────────┘        │
└──────────────────────────────────────────────────────────────────────────┘
```

특징

```
[Schema-on-Write]
- 데이터를 저장하기 전에 스키마를 정의해야 함
- 저장 시점에 데이터 검증/변환

예:
┌──────────────────────────────────────────────────────────────────────┐
│  CREATE TABLE sales (                                                 │
│      sale_id      INT PRIMARY KEY,                                    │
│      customer_id  INT NOT NULL,                                       │
│      product_id   INT NOT NULL,                                       │
│      amount       DECIMAL(10,2),                                      │
│      sale_date    DATE                                                │
│  );                                                                   │
│                                                                       │
│  -- 데이터 삽입 시 스키마 검증됨                                      │
│  INSERT INTO sales VALUES (1, 100, 50, 299.99, '2024-01-15');  │
└──────────────────────────────────────────────────────────────────────┘

[장점]
- 데이터 품질 보장 (정형화됨)
```

```
  - SQL 쿼리 최적화 → 빠른 분석
  - ACID 트랜잭션 지원
  - BI 도구와 쉽게 연동

  [단점]
  - 비정형 데이터 (이미지, 로그, JSON) 저장 어려움
  - 스키마 변경이 어려움 (경직됨)
  - 저장 비용이 비쌈
  - ETL 파이프라인 구축/유지 비용
```

대표 기술

기술	특징
Snowflake	클라우드 네이티브, 스토리지/컴퓨팅 분리
Amazon Redshift	AWS 기반, 컬럼 스토어
Google BigQuery	서버리스, 자동 스케일링
Azure Synapse	MS 생태계 통합

Part 2: Data Lake (데이터 레이크)

개념

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              Data Lake 구조                              │
│                                                                          │
│   모든 종류의 데이터                          Data Lake (저렴한 스토리지)    │
│                                                                          │
│  ┌──────────┐                                ┌──────────────────────────┐  │
│  │ 정형     │  DB 테이블, CSV              │                          │  │
│  │ 데이터   │──────────────────┐           │   /raw/                  │  │
│  └──────────┘                  │           │     ├── logs/            │  │
│                                │           │     │   └── 2024-01-15.json │  │
│                                │           │     ├── images/          │  │
│  ┌──────────┐                  │           │     │   └── product_001.jpg │  │
│  │ 반정형   │  JSON, XML, 로그  ├──────────>│     ├── transactions/    │  │
│  │ 데이터   │──────────────────┤           │     │   └── sales.parquet  │  │
│  └──────────┘                  │           │     └── videos/          │  │
│                                │           │         └── ad_001.mp4   │  │
│  ┌──────────┐                  │           │                          │  │
│  │ 비정형   │  이미지, 동영상, 오디오|       │                          │  │
│  │ 데이터   │──────────────────┘           │   그냥 다 던져 넣음!       │  │
│  └──────────┘                               │   스키마 없음              │  │
│                                             │   원본 그대로 저장         │  │
│   (스키마 검증 없이)                          └──────────────────────────┘  │
│                                                        │                 │
│                                                        │                 │
│                                                ┌──────────────┬──────────┐  │
│                                                ▼                         ▼  │
│                                        ┌──────────────┐          ┌──────────┐ │
```

```
│                                    │ Spark/EMR   │            │    ML     │ │
│                                    │ 배치 처리   │            │ Training  │ │
│                                    └─────────────┘            └───────────┘ │
│                                                                             │
└─────────────────────────────────────────────────────────────────────────────┘
```

특징

```
[Schema-on-Read]
- 저장할 때는 스키마 없음 (Raw 그대로 저장)
- 읽을 때 스키마를 적용

예:
┌──────────────────────────────────────────────────────────────┐
│  # 저장할 때: 그냥 파일로 저장                                 │
│  aws s3 cp logs.json s3://data-lake/raw/logs/                 │
│                                                               │
│  # 읽을 때: 스키마 적용                                       │
│  df = spark.read.json("s3://data-lake/raw/logs/")             │
│  df.createOrReplaceTempView("logs")                           │
│                                                               │
│  spark.sql("""                                                │
│      SELECT user_id, action, timestamp                        │
│      FROM logs                                                │
│      WHERE action = 'purchase'                                │
│  """)                                                         │
└──────────────────────────────────────────────────────────────┘

[장점]
- 모든 종류의 데이터 저장 가능
- 저장 비용이 저렴 (S3, GCS 등)
- 스키마 유연성 (나중에 정의)
- ML/AI 워크로드에 적합
- 원본 데이터 보존

[단점]
- 데이터 품질 관리 어려움 (Data Swamp 위험)
- ACID 트랜잭션 미지원
- SQL 쿼리 성능 느림
- 거버넌스/보안 복잡
```

Data Swamp (데이터 늪) 문제

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        Data Lake → Data Swamp                            │
│                                                                          │
│   초기 (깨끗한 Lake)                    시간이 지나면 (Swamp)            │
│                                                                          │
│   /data-lake/                           /data-lake/                      │
```

```
│     ├── raw/                              ├── raw/                          │
│     │   ├── sales/                        │   ├── sales/                    │
│     │   └── users/                        │   ├── sales_v2/                 │
│     └── processed/                        │   ├── sales_backup/             │
│         └── analytics/                    │   ├── users/                    │
│                                           │   ├── users_old/                │
│                                           │   ├── temp_data/                │
│   구조: 깔끔                              │   ├── test_james/               │
│   문서화: O                               │   ├── unknown_folder/           │
│   품질: 관리됨                            │   └── DO_NOT_DELETE/            │
│                                           │                                 │
│                                           │   구조: 뒤죽박죽                │
│                                           │   문서화: X                     │
│                                           │   품질: 누가 만들었는지 모름    │
│                                           │   "이 데이터 뭐야?" "몰라..."   │
│                                           │                                 │
└───────────────────────────────────────────────────────────────────────────┘
```

대표 기술

기술	특징
Amazon S3	가장 많이 사용되는 오브젝트 스토리지
Azure Data Lake Storage	Azure 생태계 통합
Google Cloud Storage	GCP 기반
HDFS	온프레미스 Hadoop 생태계

Part 3: Lakehouse (레이크하우스)

개념

```
┌───────────────────────────────────────────────────────────────────────────┐
│                          Lakehouse 구조                                   │
│                                                                           │
│   "Data Lake의 저렴함 + Data Warehouse의 기능"                            │
│                                                                           │
│   ┌─────────────────────────────────────────────────────────────────┐     │
│   │                          Lakehouse                              │     │
│   │                                                                 │     │
│   │   ┌─────────────────────────────────────────────────────────┐   │     │
│   │   │              Query Engine Layer                         │   │     │
│   │   │         (SQL, Spark, Presto, Trino)                     │   │     │
│   │   │                                                         │   │     │
│   │   │    BI 분석 ←──────────────────────→ ML/AI               │   │     │
│   │   └─────────────────────────────────────────────────────────┘   │     │
│   │                              │                                  │     │
│   │   ┌─────────────────────────────────────────────────────────┐   │     │
│   │   │           Metadata / Transaction Layer                  │   │     │
```

```
│   │   │          (Delta Lake, Iceberg, Hudi)                        │  │  │
│   │   │                                                            │  │  │
│   │   │   ☑ ACID 트랜잭션      ☑ Schema Evolution                  │  │  │
│   │   │   ☑ Time Travel        ☑ Data Versioning                   │  │  │
│   │   │   ☑ Unified Batch/Stream                                   │  │  │
│   │   └────────────────────────────────────────────────────────────┘  │  │
│   │                              │                                    │  │
│   │   ┌────────────────────────────────────────────────────────────┐  │  │
│   │   │             Storage Layer (저렴한 오브젝트 스토리지)          │  │  │
│   │   │                  (S3, ADLS, GCS)                           │  │  │
│   │   │                                                            │  │  │
│   │   │   Parquet, ORC 파일들 (오픈 포맷)                           │  │  │
│   │   └────────────────────────────────────────────────────────────┘  │  │
│   │                                                                   │  │
│   └───────────────────────────────────────────────────────────────────┘  │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

핵심: 테이블 포맷 (Delta Lake, Iceberg, Hudi)

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    Delta Lake 동작 원리 예시                              │
│                                                                          │
│   기존 Data Lake:                      Delta Lake:                       │
│   (그냥 파일들)                         (트랜잭션 로그 추가)               │
│                                                                          │
│   /sales/                              /sales/                           │
│     ├── part-001.parquet               ├── _delta_log/                   │
│     ├── part-002.parquet               │   ├── 00000.json  (v0: 초기)     │
│     └── part-003.parquet               │   ├── 00001.json  (v1: INSERT)   │
│                                        │   ├── 00002.json  (v2: UPDATE)   │
│   문제:                                │   └── 00003.json  (v3: DELETE)   │
│   - 동시 쓰기 충돌                      ├── part-001.parquet               │
│   - 부분 실패 시 불일치                 ├── part-002.parquet               │
│   - 이전 버전 복구 불가                 └── part-003.parquet               │
│                                                                          │
│                                        해결:                             │
│                                        - ACID 트랜잭션                    │
│                                        - Time Travel (과거 버전 조회)      │
│                                        - Schema Enforcement              │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘

[Time Travel 예시]
┌──────────────────────────────────────────────────────────────────────┐
│  -- 현재 데이터 조회                                                  │
│  SELECT * FROM sales;                                                 │
│                                                                       │
│  -- 어제 시점의 데이터 조회 (Time Travel)                              │
│  SELECT * FROM sales VERSION AS OF 2;                                 │
│  SELECT * FROM sales TIMESTAMP AS OF '2024-01-14';                    │
```

```
│                                                                │
│  -- 실수로 삭제한 데이터 복구                                  │
│  RESTORE TABLE sales TO VERSION AS OF 1;                       │
└────────────────────────────────────────────────────────────────┘
```

Lakehouse의 핵심 기능

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          Lakehouse 핵심 기능                             │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  1. ACID Transactions (트랜잭션)                                         │
│  ──────────────────────────────────                                      │
│  - 동시에 여러 작업이 데이터 수정해도 안전                               │
│  - 실패 시 롤백 가능                                                     │
│  - Data Lake에서 불가능했던 UPDATE, DELETE, MERGE 지원                   │
│                                                                          │
│  2. Schema Evolution (스키마 진화)                                       │
│  ──────────────────────────────────                                      │
│  - 컬럼 추가/삭제/이름변경 유연하게 가능                                 │
│  - 기존 데이터와 호환성 유지                                             │
│                                                                          │
│  3. Time Travel (시간 여행)                                              │
│  ──────────────────────────────────                                      │
│  - 과거 특정 시점의 데이터 조회                                          │
│  - 실수로 삭제/변경한 데이터 복구                                        │
│  - 감사(Audit) 목적으로 히스토리 추적                                    │
│                                                                          │
│  4. Unified Batch & Streaming                                            │
│  ──────────────────────────────────                                      │
│  - 같은 테이블에 배치와 스트리밍 동시 처리                               │
│  - 실시간 데이터와 과거 데이터 통합 분석                                 │
│                                                                          │
│  5. BI + ML 통합                                                         │
│  ──────────────────────────────────                                      │
│  - BI 도구에서 SQL로 직접 조회 (Warehouse처럼)                           │
│  - ML 프레임워크에서 직접 학습 데이터로 사용 (Lake처럼)                  │
│  - 데이터 복사/이동 불필요                                               │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

대표 기술

기술	개발사	특징
Delta Lake	Databricks	Spark와 긴밀한 통합, 가장 많이 사용
Apache Iceberg	Netflix/Apple	벤더 중립적, 대규모 테이블에 강함
Apache Hudi	Uber	스트리밍/증분 처리에 특화

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          테이블 포맷 비교                                  │
├──────────────────┬──────────────────┬──────────────────┬──────────────────┤
│                  │   Delta Lake     │    Iceberg       │       Hudi       │
├──────────────────┼──────────────────┼──────────────────┼──────────────────┤
│ 개발사           │ Databricks       │ Netflix/Apple    │ Uber             │
├──────────────────┼──────────────────┼──────────────────┼──────────────────┤
│ 주요 강점        │ Spark 통합       │ 대규모 테이블    │ 스트리밍/증분 처리 │
│                  │ 커뮤니티 큼      │ 벤더 중립        │ Upsert 최적화    │
├──────────────────┼──────────────────┼──────────────────┼──────────────────┤
│ 쿼리 엔진        │ Spark, Trino,    │ Spark, Trino,    │ Spark, Hive,     │
│                  │ Presto, Flink    │ Flink, Presto    │ Presto           │
├──────────────────┼──────────────────┼──────────────────┼──────────────────┤
│ Time Travel      │ ☑                │ ☑                │ ☑                │
├──────────────────┼──────────────────┼──────────────────┼──────────────────┤
│ ACID             │ ☑                │ ☑                │ ☑                │
├──────────────────┼──────────────────┼──────────────────┼──────────────────┤
│ 스트리밍         │ ☑                │ ☑ (제한적)       │ ☑ (특화)         │
└──────────────────┴──────────────────┴──────────────────┴──────────────────┘
```

Part 4: 아키텍처 비교

전통적 구조 vs Lakehouse

```
┌──────────────────────────────────────────────────────────────────────────┐
│                 전통적 구조: Data Lake + Data Warehouse 분리              │
│                                                                          │
│                          ┌─────────────────┐                             │
│                          │  Data Sources   │                             │
│                          └─────────────────┘                             │
│                                   │                                      │
│                                   ▼                                      │
│                          ┌─────────────────┐                             │
│                          │   Data Lake     │  (Raw 저장)                 │
│                          │   (S3, ADLS)    │                             │
│                          └─────────────────┘                             │
│                                   │                                      │
│                     ┌─────────────────────────┐                          │
│                     │          ETL            │  (데이터 복사!)          │
│                     │   (비용 + 지연 발생)    │                          │
│                     └─────────────────────────┘                          │
│                                   │                                      │
│                   ┌───────────────┴───────────────┐                      │
│                   ▼                               ▼                      │
│          ┌─────────────────┐                ┌─────────────────┐          │
│          │ Data Warehouse  │                │   ML Platform   │          │
│          │ (Snowflake 등)  │                │ (Databricks 등) │          │
│          └─────────────────┘                └─────────────────┘          │
│                   │                               │                      │
│                   │                               │                      │
```

```
│              ▼                                   ▼                    │
│      ┌──────────────────┐                ┌──────────────────┐        │
│      │       BI         │                │       ML         │        │
│      │   Dashboard      │                │    Training      │        │
│      └──────────────────┘                └──────────────────┘        │
│                                                                       │
│      문제점:                                                          │
│      - 데이터 중복 저장 (비용 증가)                                   │
│      - ETL 파이프라인 복잡                                            │
│      - 데이터 불일치 가능성                                           │
│      - 실시간 분석 어려움                                             │
│                                                                       │
└───────────────────────────────────────────────────────────────────────┘


┌───────────────────────────────────────────────────────────────────────┐
│                     Lakehouse: 통합 구조                              │
│                                                                       │
│                          ┌──────────────────┐                         │
│                          │  Data Sources    │                         │
│                          └──────────────────┘                         │
│                                   │                                   │
│                                   ▼                                   │
│      ┌──────────────────────────────────────────────────────────┐     │
│      │                       Lakehouse                          │     │
│      │   ┌──────────────────────────────────────────────────┐   │     │
│      │   │              Delta Lake / Iceberg                 │   │     │
│      │   │           (ACID + Time Travel + Schema)           │   │     │
│      │   └──────────────────────────────────────────────────┘   │     │
│      │                           │                              │     │
│      │   ┌──────────────────────────────────────────────────┐   │     │
│      │   │             Object Storage (S3, ADLS)             │   │     │
│      │   │                  (저렴한 저장)                    │   │     │
│      │   └──────────────────────────────────────────────────┘   │     │
│      └──────────────────────────────────────────────────────────┘     │
│                                   │                                   │
│              ┌────────────────────┴────────────────────┐              │
│              │        (ETL 없이 직접 접근!)            │              │
│              ▼                                         ▼              │
│      ┌──────────────────┐                ┌──────────────────┐        │
│      │       BI         │                │       ML         │        │
│      │   (SQL 쿼리)     │                │   (DataFrame)    │        │
│      └──────────────────┘                └──────────────────┘        │
│                                                                       │
│      장점:                                                            │
│      - 단일 데이터 소스 (Single Source of Truth)                      │
│      - 데이터 복사 불필요                                             │
│      - 실시간 분석 가능                                               │
│      - 비용 절감                                                      │
│                                                                       │
└───────────────────────────────────────────────────────────────────────┘
```

Part 5: 언제 무엇을 사용?

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              선택 가이드                                  │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  [Data Warehouse 선택]                                                   │
│  ────────────────────                                                    │
│  ☑ 정형 데이터만 다룸                                                    │
│  ☑ BI/리포팅이 주요 목적                                                 │
│  ☑ SQL 분석이 대부분                                                     │
│  ☑ 데이터 품질/거버넌스가 최우선                                         │
│  ☑ 팀에 SQL 사용자가 많음                                                │
│                                                                          │
│  추천: Snowflake, BigQuery, Redshift                                     │
│                                                                          │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  [Data Lake 선택]                                                        │
│  ────────────────────                                                    │
│  ☑ 비정형 데이터가 많음 (이미지, 로그, 동영상)                           │
│  ☑ ML/AI가 주요 목적                                                     │
│  ☑ 저장 비용이 중요                                                      │
│  ☑ 원본 데이터 보존 필요                                                 │
│  ☑ 스키마가 자주 변경됨                                                  │
│                                                                          │
│  추천: S3 + Spark/EMR, Azure Data Lake + Databricks                      │
│                                                                          │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  [Lakehouse 선택] ★ 권장                                                 │
│  ────────────────────                                                    │
│  ☑ BI와 ML 둘 다 필요                                                    │
│  ☑ 실시간 + 배치 처리 통합                                               │
│  ☑ 데이터 복사/이동 최소화                                               │
│  ☑ 비용 효율 + 데이터 품질 둘 다 중요                                    │
│  ☑ 새로 시작하는 프로젝트                                                │
│                                                                          │
│  추천: Databricks (Delta Lake), AWS (Iceberg), Tabular (Iceberg)         │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

정리

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              최종 비교표                                  │
├──────────────────┬──────────────────┬──────────────────┬──────────────────┤
│                  │ Data Warehouse   │   Data Lake      │    Lakehouse     │
├──────────────────┼──────────────────┼──────────────────┼──────────────────┤
```

```
│ 데이터 형태    │ 정형만          │ 모두            │ 모두                    │
├──────────────┼────────────────┼────────────────┼────────────────────────┤
│ 스키마         │ Schema-on-Write │ Schema-on-Read  │ 둘 다                   │
├──────────────┼────────────────┼────────────────┼────────────────────────┤
│ 저장 비용      │ $$$             │ $               │ $                       │
├──────────────┼────────────────┼────────────────┼────────────────────────┤
│ ACID           │ ☑               │ ❌              │ ☑                       │
├──────────────┼────────────────┼────────────────┼────────────────────────┤
│ Time Travel    │ 일부            │ ❌              │ ☑                       │
├──────────────┼────────────────┼────────────────┼────────────────────────┤
│ 쿼리 성능      │ 빠름            │ 느림            │ 빠름                    │
├──────────────┼────────────────┼────────────────┼────────────────────────┤
│ BI 지원        │ ☑ 최적화        │ 제한적          │ ☑                       │
├──────────────┼────────────────┼────────────────┼────────────────────────┤
│ ML 지원        │ 제한적          │ ☑ 최적화        │ ☑                       │
├──────────────┼────────────────┼────────────────┼────────────────────────┤
│ 실시간 처리    │ 제한적          │ ☑               │ ☑                       │
├──────────────┼────────────────┼────────────────┼────────────────────────┤
│ 주요 사용자    │ BI 분석가       │ 데이터 과학자   │ 모두                    │
├──────────────┼────────────────┼────────────────┼────────────────────────┤
│ 대표 기술      │ Snowflake       │ S3 + Spark      │ Delta Lake, Iceberg     │
│                │ Redshift        │ HDFS            │ Hudi                    │
│                │ BigQuery        │                 │                         │
└──────────────┴────────────────┴────────────────┴────────────────────────┘

요약:
- Warehouse: "깔끔하고 빠른 SQL 분석" (비쌈)
- Lake: "뭐든 저장, 나중에 처리" (지저분해질 수 있음)
- Lakehouse: "Lake의 저렴함 + Warehouse의 기능" (현재 트렌드)
```